Assignment 1: Sorting Race Results with Two Algorithms

Saduakhas Olzhas - 26MD0341

1. Implementation and Verification

I wrote this sorting tool in Python 3.13.7. Neither sorting function calls built-in sort() or sorted() inside its code. Each race finish time is just a whole number of milliseconds between 0 and 9,999,999. Before running a sort, my script copies the input list with list(original_data). The original list never changes. That way both algorithms get the exact same unsorted numbers every time.

My Merge Sort code follows the MergeSort and MergeLists pseudocode from Lecture 3. First, _merge_sort_recursive finds the middle index with (first + last) // 2 and splits the list in half. It keeps splitting until first is at least last. A slice of 0 or 1 item is already sorted. After that, merge_lists combines the two halves into a temporary list called temp_result and copies the sorted range back into arr. If the left value is less than or equal to the right value, it picks the left one first so duplicate times stay in their original order.

For LSD Radix Sort in base 10, I followed the bucket algorithm from Lecture 2 on Slide 44. Since the largest finish time is 9,999,999 ms, a number can have at most M = 7 digits. So the outer loop always runs 7 times. The shift variable starts at 1 and gets multiplied by 10 on each pass up to 1,000,000. On pass k, each number goes into one of ten buckets from 0 to 9 using (val // shift) % 10. When shift is bigger than the number itself, like 45 on the hundreds pass, integer division gives 0. This handles leading zeroes right away without turning numbers into strings. At the end of each pass, buckets 0 to 9 are copied back into arr in order, which keeps the sort stable.

Before timing the big datasets, I ran six small tests that I could check by hand. These covered an empty list, a single number, a short mixed list, repeated times, an already sorted list, and a reverse-ordered list. My verify_sort_output helper checks three things after every run. First, it checks that the input list did not change. Second, is_ascending loops through the output to make sure no item is bigger than the next one. Third, it checks that both outputs have the exact same values as the input and match each other at every position. All checks passed as shown in Table 1.

Table 1
Hand-Checkable Correctness Verification Summary

Test Case	Input List	Merge Sort Output	LSD Radix Sort Output	Status
Empty list	[]	[]	[]	PASS
One-value list	[42050]	[42050]	[42050]	PASS
Short hand-checked	[125000, 45, 9999999, 0, 305, 45, 7]	[0, 7, 45, 45, 305, 125000, 9999999]	[0, 7, 45, 45, 305, 125000, 9999999]	PASS
Repeated values	[500, 120, 500, 120, 0, 0, 500, 120]	[0, 0, 120, 120, 120, 500, 500, 500]	[0, 0, 120, 120, 120, 500, 500, 500]	PASS
Already sorted	[0, 15, 240, 1024, 50000, 1200000, 9999999]	[0, 15, 240, 1024, 50000, 1200000, 9999999]	[0, 15, 240, 1024, 50000, 1200000, 9999999]	PASS
Reverse-ordered	[9999999, 850000, 40000, 3200, 150, 9, 0]	[0, 9, 150, 3200, 40000, 850000, 9999999]	[0, 9, 150, 3200, 40000, 850000, 9999999]	PASS

2. Race Data and Benchmark Setup

To make the test data reproducible, I set a fixed seed of 2026 in random.Random(2026). Then I created five random lists of finish times between 0 and 9,999,999 ms. The five list sizes are 1,000, 5,000, 10,000, 50,000, and 100,000 records. If anyone runs my script with seed 2026, it will generate the exact same random numbers and operation counts.

I ran all tests on my laptop with a 13th Gen Intel Core i5-1335U processor at 1.30 GHz and 16 GB of RAM on Windows 11. The code ran on 64-bit Python 3.13.7. To measure time, I called time.perf_counter() right before and right after the sorting function and multiplied the difference by 1000 to get milliseconds. Generating data, copying lists, printing, and drawing charts all happen outside the timer. I ran each sort 3 times for every size N and recorded the median time.

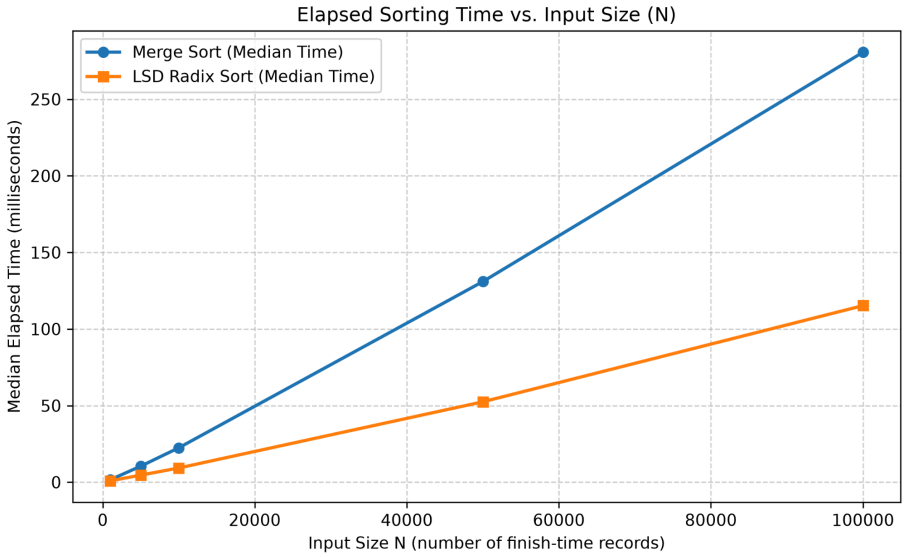
Because the two sorts do different kinds of work, their operation counters are tracked separately. In Merge Sort, key_comparisons adds 1 each time two elements from the left and right halves are compared in the merge loop, and element_copies adds 1 per write into temp_result plus 1 per write back into arr. In LSD Radix Sort, digit_inspections adds 1 per (val // shift) % 10 digit calculation, and element_placements adds 1 per bucket append plus 1 per write back into arr during bucket combination.

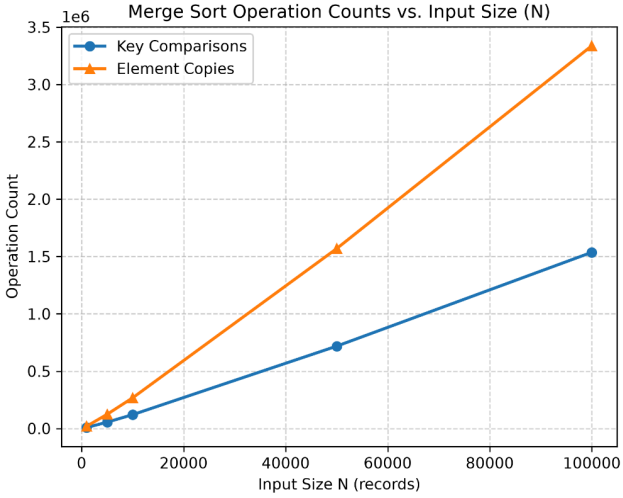
3. Results Table and Charts

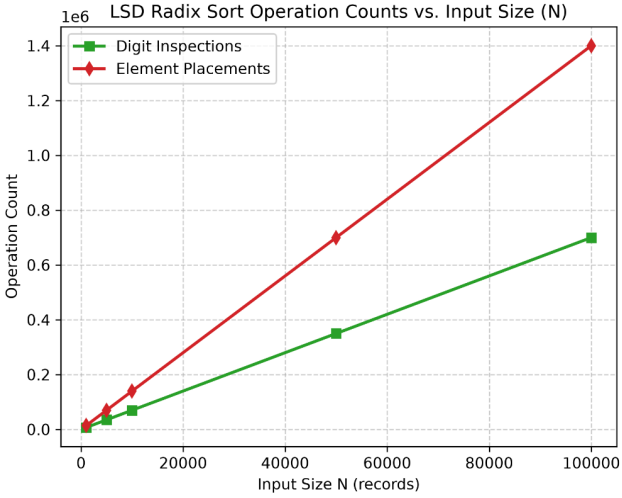
Table 2 shows the median time over 3 runs and the operation counts for all five input sizes.

Table 2
Measured Performance and Basic Operation Counts (N = 1,000 to 100,000)

Input Size (N)	Merge Sort Median Time (ms)	Merge Sort Key Comparisons	Merge Sort Element Copies	Radix Sort Median Time (ms)	Radix Sort Digit Inspections	Radix Sort Element Placements
1,000	1.617	8,717	19,952	0.903	7,000	14,000
5,000	10.473	55,209	123,616	4.640	35,000	70,000
10,000	22.467	120,462	267,232	9.290	70,000	140,000
50,000	130.967	718,151	1,568,928	52.492	350,000	700,000
100,000	280.670	1,536,433	3,337,856	115.270	700,000	1,400,000







4. Performance Analysis

In Lecture 3 on Slides 4 to 10, finding the middle index takes 0 comparisons. All comparisons happen inside MergeLists when merging two sorted halves of total size N. If every number in the left half is smaller than the first number in the right half, MergeLists only needs N / 2 comparisons. That is the best case. Using repeated substitution on B(N) = 2B(N/2) + N/2 with B(0) = B(1) = 0 gives B(N) = (N lg N) / 2, which is O(N lg N). The worst case happens when values from the two halves take turns on every step. Then merging needs N - 1 comparisons. Solving W(N) = 2W(N/2) + N - 1 by repeated substitution across lg N levels on Slides 7 to 9 gives W(N) = N lg N - N + 1. That is also O(N lg N). Since any random input takes between (N lg N) / 2 and N lg N - N + 1 comparisons, the average case is O(N lg N) too, and all three cases are Theta(N lg N).

Unlike Merge Sort, LSD Radix Sort never compares two elements against each other. Per Lecture 2 on Slides 44 to 46, an outer loop runs M = keySize passes for the number of digits in the longest key, and an inner loop goes through all N records. Finish times in 0 to 9,999,999 have at most M = 7 decimal digits. Each pass checks 1 digit per record and moves each record twice, once into a bucket and once back into the array. The loop counts only depend on M and N, never on the order of the input. So the best case, average case, and worst case are all O(M * N) and Theta(M * N). With a fixed range where M = 7 is small compared to N, Lecture 2 simplifies this to linear O(N). Still, the key range sets the constant M. Switching from 7-digit milliseconds to 16-digit nanosecond keys would raise M from 7 to 16 and more than double the passes for the same N.

My measured numbers match the lecture formulas really well. For N = 100,000, lg N is about 16.61. If we plug that into the Lecture 3 equations, the best case B(100,000) is 830,482 comparisons and the worst case W(100,000) is 1,560,965 comparisons. My run on random data counted 1,536,433 comparisons. That sits right inside the range and close to the top because two random halves usually interleave until both are almost empty. Also, when N goes from 1,000 to 100,000, N grows by 100 times and theoretical N lg N grows by 166.7 times. In my table, Merge Sort comparisons grew by 176.26 times from 8,717 to 1,536,433. Its median time grew by 173.57 times from 1.617 ms to 280.670 ms. For LSD Radix Sort, the operation counts follow a straight line. When N grows by 100 times, digit inspections grow by 100 times from 7,000 to 700,000 (which is 7N) and placements grow by 100 times from 14,000 to 1,400,000 (which is 14N). Its median time went from 0.903 ms to 115.270 ms, or 127.65 times. That is a bit over 100 times because larger bucket lists need more memory allocation at N = 100,000.

Comparing the two sets of counters shows the main trade-off between comparing keys and moving data in memory. On the 100,000-record run, Merge Sort did 1,536,433 comparisons, but it had to do 3,337,856 element copies using an extra buffer of size N. Radix Sort did 0 comparisons, but as Lecture 2 points out on Slide 46, bucket arrays need extra memory and move every record 2M = 14 times. While LSD Radix Sort ran about 2.4 times faster than Merge Sort at N = 100,000 on my laptop (115.270 ms vs. 280.670 ms), one dataset on one computer does not mean Radix Sort is always faster. On small lists with wide 64-bit keys where M is large, floating-point numbers, or big records where moving items 14 times is slow, Merge Sort can beat Radix Sort.

5. Reflection

Formulas like O(N lg N) and O(M * N) in the lecture notes made me focus almost entirely on key comparisons. Adding counters to the code and timing both sorts showed me how much buffer copies and constants matter in practice. When I ran Merge Sort on 100,000 records, it did 1,536,433 key comparisons, but it did 3,337,856 element copies. Seeing more than two array copies for every key comparison explained why copying values into temp_result and back into arr takes up so much of the runtime in Merge Sort.

Measuring LSD Radix Sort also showed me how the digit count M = 7 affects different input sizes. In theory, O(N) sounds like it should beat O(N lg N) by a lot on any list size. On the N = 1,000 test, though, Radix Sort still did 7,000 digit inspections and 14,000 placements for 21,000 operations in total, since each item moves 2M = 14 times. Merge Sort did 8,717 comparisons and 19,952 copies, or 28,669 operations in total. Because Python had to create ten bucket lists across 7 passes, Radix Sort took 0.903 ms vs. 1.617 ms for Merge Sort at N = 1,000, which is less than a 2x gap. Only when N hit 100,000 did the lg N = 16.61 factor push Merge Sort to 280.670 ms against 115.270 ms for Radix Sort. Big-Oh gives the growth trend, but data movement and M decide the crossover.

Appendix A: Console Test Evidence and Benchmark Log

```
================================================================================
STEP 1: CORRECTNESS VERIFICATION ON HAND-CHECKABLE TEST CASES
================================================================================
[PASS] Empty list
       Input       : []
       Merge Output: []
       Radix Output: []
[PASS] One-value list
       Input       : [42050]
       Merge Output: [42050]
       Radix Output: [42050]
[PASS] Short hand-checked list
       Input       : [125000, 45, 9999999, 0, 305, 45, 7]
       Merge Output: [0, 7, 45, 45, 305, 125000, 9999999]
       Radix Output: [0, 7, 45, 45, 305, 125000, 9999999]
[PASS] Repeated values
       Input       : [500, 120, 500, 120, 0, 0, 500, 120]
       Merge Output: [0, 0, 120, 120, 120, 500, 500, 500]
       Radix Output: [0, 0, 120, 120, 120, 500, 500, 500]
[PASS] Already sorted values
       Input       : [0, 15, 240, 1024, 50000, 1200000, 9999999]
       Merge Output: [0, 15, 240, 1024, 50000, 1200000, 9999999]
       Radix Output: [0, 15, 240, 1024, 50000, 1200000, 9999999]
[PASS] Reverse-ordered values
       Input       : [9999999, 850000, 40000, 3200, 150, 9, 0]
       Merge Output: [0, 9, 150, 3200, 40000, 850000, 9999999]
       Radix Output: [0, 9, 150, 3200, 40000, 850000, 9999999]
--------------------------------------------------------------------------------
All hand-checked correctness tests passed successfully.

================================================================================
ENVIRONMENT & SYSTEM DETAILS
================================================================================
OS / Platform       : Windows 11 (10.0.26200)
Processor / Machine : Intel64 Family 6 Model 186 Stepping 3, GenuineIntel / AMD64
                      (13th Gen Intel(R) Core(TM) i5-1335U @ 1.30 GHz, 16.0 GB RAM)
Python Version      : 3.13.7
Timer Function      : time.perf_counter() (high-resolution monotonic clock)
Random Seed         : 2026
================================================================================

================================================================================
STEP 2 & 3: BENCHMARKING ON RACE DATASETS (MEDIAN OF 3 RUNS)
================================================================================
N =   1,000 | VERIFIED OK | Merge Median:    1.617 ms (Comp: 8,717, Copies: 19,952) | Radix Median:
0.903 ms (Digit Insp: 7,000, Placements: 14,000)
N =   5,000 | VERIFIED OK | Merge Median:   10.473 ms (Comp: 55,209, Copies: 123,616) | Radix Median:
4.640 ms (Digit Insp: 35,000, Placements: 70,000)
N =  10,000 | VERIFIED OK | Merge Median:   22.467 ms (Comp: 120,462, Copies: 267,232) | Radix Median:
9.290 ms (Digit Insp: 70,000, Placements: 140,000)
N =  50,000 | VERIFIED OK | Merge Median:  130.967 ms (Comp: 718,151, Copies: 1,568,928) | Radix
Median:   52.492 ms (Digit Insp: 350,000, Placements: 700,000)
N = 100,000 | VERIFIED OK | Merge Median:  280.670 ms (Comp: 1,536,433, Copies: 3,337,856) | Radix
Median:  115.270 ms (Digit Insp: 700,000, Placements: 1,400,000)
```

Appendix B: Source Code

```
import copy
import platform
import random
import statistics
import sys
import time
import matplotlib.pyplot as plt

# Fixed seed so every generated dataset can be reproduced exactly
RANDOM_SEED = 2026
MIN_TIME_MS = 0
MAX_TIME_MS = 9_999_999
MAX_DIGITS = 7  # 0 through 9,999,999 has at most 7 decimal digits
INPUT_SIZES = [1_000, 5_000, 10_000, 50_000, 100_000]
TRIALS_PER_SIZE = 3


class MergeSortMetrics:
    """Tracks basic operations performed by Merge Sort."""
    def __init__(self):
        self.key_comparisons = 0
        self.element_copies = 0

    @property
    def total_operations(self):
        return self.key_comparisons + self.element_copies


class RadixSortMetrics:
    """Tracks basic operations performed by LSD Radix Sort."""
    def __init__(self):
        self.digit_inspections = 0
        self.element_placements = 0

    @property
    def total_operations(self):
        return self.digit_inspections + self.element_placements


# ======================================================================
# 1. MERGE SORT IMPLEMENTATION
# ======================================================================

def merge_lists(arr, start1, end1, start2, end2, temp_result, metrics):
    """
    Merges two contiguous sorted sublists arr[start1..end1] and arr[start2..end2]
    into temp_result and copies the merged range back into arr.
    """
    final_start = start1
    final_end = end2
    index_c = final_start

    while start1 <= end1 and start2 <= end2:
        metrics.key_comparisons += 1
        # Using <= preserves stability for equal finish times
        if arr[start1] <= arr[start2]:
            temp_result[index_c] = arr[start1]
            start1 += 1
        else:
            temp_result[index_c] = arr[start2]
            start2 += 1
        metrics.element_copies += 1
        index_c += 1

    # Move the part of the sublist that is left over
```

```
    if start1 <= end1:
        for i in range(start1, end1 + 1):
            temp_result[index_c] = arr[i]
            metrics.element_copies += 1
            index_c += 1
    else:
        for i in range(start2, end2 + 1):
            temp_result[index_c] = arr[i]
            metrics.element_copies += 1
            index_c += 1

    # Copy merged elements back into original list segment
    for i in range(final_start, final_end + 1):
        arr[i] = temp_result[i]
        metrics.element_copies += 1


def _merge_sort_recursive(arr, first, last, temp_result, metrics):
    """Recursive divide-and-conquer helper matching MergeSort(list, first, last)."""
    if first < last:
        middle = (first + last) // 2
        _merge_sort_recursive(arr, first, middle, temp_result, metrics)
        _merge_sort_recursive(arr, middle + 1, last, temp_result, metrics)
        merge_lists(arr, first, middle, middle + 1, last, temp_result, metrics)


def merge_sort(data):
    """
    Entry point for Merge Sort.
    Preserves original input by sorting a working copy.
    Returns (sorted_list, MergeSortMetrics).
    """
    arr = list(data)
    metrics = MergeSortMetrics()
    n = len(arr)
    if n > 1:
        temp_result = [0] * n
        _merge_sort_recursive(arr, 0, n - 1, temp_result, metrics)
    return arr, metrics


# ============================================================================
# 2. LSD RADIX SORT IMPLEMENTATION
# ============================================================================

def radix_sort_lsd(data, key_size=MAX_DIGITS):
    """
    Stable Least-Significant-Digit (LSD) Radix Sort in base 10.
    Handles leading zeroes implicitly via integer division and modulo 10.
    Returns (sorted_list, RadixSortMetrics).
    """
    arr = list(data)
    metrics = RadixSortMetrics()
    n = len(arr)
    if n <= 1:
        return arr, metrics

    shift = 1
    for _ in range(key_size):
        buckets = [[] for _ in range(10)]

        # Distribute entries into base-10 buckets by current digit
        for entry in range(n):
            val = arr[entry]
            bucket_number = (val // shift) % 10
            metrics.digit_inspections += 1
```

```
            buckets[bucket_number].append(val)
            metrics.element_placements += 1

        # CombineBuckets(): recombine buckets 0..9 in order to preserve stability
        idx = 0
        for b in range(10):
            for val in buckets[b]:
                arr[idx] = val
                metrics.element_placements += 1
                idx += 1

        shift *= 10

    return arr, metrics


# ============================================================================
# 3. CORRECTNESS VERIFICATION
# ============================================================================

def is_ascending(arr):
    """Checks that every element is <= the next element."""
    for i in range(len(arr) - 1):
        if arr[i] > arr[i + 1]:
            return False
    return True


def verify_sort_output(original, sorted_merge, sorted_radix):
    """
    Verifies that both outputs are in ascending order, match each other,
    and contain the exact same multiset of values as the original input.
    """
    expected = sorted(original)
    merge_ok = is_ascending(sorted_merge) and (sorted_merge == expected)
    radix_ok = is_ascending(sorted_radix) and (sorted_radix == expected)
    same_output = (sorted_merge == sorted_radix)
    return merge_ok and radix_ok and same_output


def run_correctness_suite():
    """Runs hand-checkable test cases required before benchmarking."""
    test_cases = [
        ("Empty list", []),
        ("One-value list", [42050]),
        ("Short hand-checked list", [125000, 45, 9999999, 0, 305, 45, 7]),
        ("Repeated values", [500, 120, 500, 120, 0, 0, 500, 120]),
        ("Already sorted values", [0, 15, 240, 1024, 50000, 1200000, 9999999]),
        ("Reverse-ordered values", [9999999, 850000, 40000, 3200, 150, 9, 0]),
    ]

    print("=" * 75)
    print("STEP 1: CORRECTNESS VERIFICATION ON HAND-CHECKABLE TEST CASES")
    print("=" * 75)

    for name, dataset in test_cases:
        input_snapshot = copy.deepcopy(dataset)
        out_merge, _ = merge_sort(dataset)
        out_radix, _ = radix_sort_lsd(dataset)

        # Ensure original input was not mutated
        input_preserved = (dataset == input_snapshot)
        passed = verify_sort_output(dataset, out_merge, out_radix) and input_preserved

        status = "PASS" if passed else "FAIL"
        print(f"[{status}] {name}")
        print(f"       Input       : {dataset}")
```

```
        print(f"       Merge Output: {out_merge}")
        print(f"       Radix Output: {out_radix}")
        if not passed:
            raise AssertionError(f"Correctness check failed on {name}")
    print("-" * 75)
    print("All hand-checked correctness tests passed successfully.\n")


# ======================================================================
# 4. BENCHMARKING & CHART GENERATION
# ======================================================================

def run_benchmarks():
    print("=" * 75)
    print("ENVIRONMENT & SYSTEM DETAILS")
    print("=" * 75)
    print(f"OS / Platform       : {platform.system()} {platform.release()} ({platform.version()})")
    print(f"Processor / Machine : {platform.processor()} / {platform.machine()}")
    print(f"Python Version      : {sys.version.split()[0]}")
    print(f"Timer Function      : time.perf_counter() (high-resolution monotonic clock)")
    print(f"Random Seed         : {RANDOM_SEED}")
    print("=" * 75 + "\n")

    rng = random.Random(RANDOM_SEED)
    results = []

    print("=" * 75)
    print("STEP 2 & 3: BENCHMARKING ON RACE DATASETS (MEDIAN OF 3 RUNS)")
    print("=" * 75)

    for n in INPUT_SIZES:
        # Generate one random dataset for this size
        original_data = [rng.randint(MIN_TIME_MS, MAX_TIME_MS) for _ in range(n)]

        merge_times_ms = []
        radix_times_ms = []
        merge_metrics_final = None
        radix_metrics_final = None

        # 3 trials for Merge Sort
        for trial in range(TRIALS_PER_SIZE):
            work_copy = list(original_data)
            t0 = time.perf_counter()
            sorted_m, m_metrics = merge_sort(work_copy)
            t1 = time.perf_counter()
            merge_times_ms.append((t1 - t0) * 1000.0)
            if trial == 0:
                merge_metrics_final = m_metrics
                saved_merge_out = sorted_m

        # 3 trials for LSD Radix Sort
        for trial in range(TRIALS_PER_SIZE):
            work_copy = list(original_data)
            t0 = time.perf_counter()
            sorted_r, r_metrics = radix_sort_lsd(work_copy)
            t1 = time.perf_counter()
            radix_times_ms.append((t1 - t0) * 1000.0)
            if trial == 0:
                radix_metrics_final = r_metrics
                saved_radix_out = sorted_r

        # Verify correctness on every generated race dataset
        assert verify_sort_output(original_data, saved_merge_out, saved_radix_out), (
            f"Verification failed for N={n}"
        )

        med_merge_ms = statistics.median(merge_times_ms)
```

```
        med_radix_ms = statistics.median(radix_times_ms)

        row = {
            "n": n,
            "merge_time_ms": med_merge_ms,
            "merge_trials_ms": merge_times_ms,
            "merge_comparisons": merge_metrics_final.key_comparisons,
            "merge_copies": merge_metrics_final.element_copies,
            "merge_total_ops": merge_metrics_final.total_operations,
            "radix_time_ms": med_radix_ms,
            "radix_trials_ms": radix_times_ms,
            "radix_inspections": radix_metrics_final.digit_inspections,
            "radix_placements": radix_metrics_final.element_placements,
            "radix_total_ops": radix_metrics_final.total_operations,
        }
        results.append(row)

        print(
            f"N = {n:7,d} | VERIFIED OK | "
            f"Merge Median: {med_merge_ms:8.3f} ms "
            f"(Comp: {row['merge_comparisons']:,}, Copies: {row['merge_copies']:,}) | "
            f"Radix Median: {med_radix_ms:8.3f} ms "
            f"(Digit Insp: {row['radix_inspections']:,}, Placements: {row['radix_placements']:,})"
        )

    return results


def generate_charts(results):
    sizes = [r["n"] for r in results]
    merge_times = [r["merge_time_ms"] for r in results]
    radix_times = [r["radix_time_ms"] for r in results]

    merge_comps = [r["merge_comparisons"] for r in results]
    merge_copies = [r["merge_copies"] for r in results]
    radix_insp = [r["radix_inspections"] for r in results]
    radix_place = [r["radix_placements"] for r in results]

    # Chart 1: Elapsed Time vs Input Size
    plt.figure(figsize=(8, 5), dpi=300)
    plt.plot(sizes, merge_times, marker="o", linewidth=2, label="Merge Sort (Median Time)")
    plt.plot(sizes, radix_times, marker="s", linewidth=2, label="LSD Radix Sort (Median Time)")
    plt.title("Elapsed Sorting Time vs. Input Size (N)")
    plt.xlabel("Input Size N (number of finish-time records)")
    plt.ylabel("Median Elapsed Time (milliseconds)")
    plt.grid(True, linestyle="--", alpha=0.6)
    plt.legend()
    plt.tight_layout()
    plt.savefig("elapsed_time_chart.png")
    plt.close()

    # Chart 2: Operation Counts vs Input Size (Side-by-Side Subplots)
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5), dpi=300)

    ax1.plot(sizes, merge_comps, marker="o", linewidth=2, label="Key Comparisons")
    ax1.plot(sizes, merge_copies, marker="^", linewidth=2, label="Element Copies")
    ax1.set_title("Merge Sort Operation Counts vs. Input Size (N)")
    ax1.set_xlabel("Input Size N (records)")
    ax1.set_ylabel("Operation Count")
    ax1.grid(True, linestyle="--", alpha=0.6)
    ax1.legend()

    ax2.plot(sizes, radix_insp, marker="s", linewidth=2, color="tab:green", label="Digit Inspections")
    ax2.plot(sizes, radix_place, marker="d", linewidth=2, color="tab:red", label="Element Placements")
    ax2.set_title("LSD Radix Sort Operation Counts vs. Input Size (N)")
    ax2.set_xlabel("Input Size N (records)")
    ax2.set_ylabel("Operation Count")
```

```
    ax2.grid(True, linestyle="--", alpha=0.6)
    ax2.legend()

    plt.tight_layout()
    plt.savefig("operation_counts_chart.png")
    plt.close()
    print("\nCharts saved successfully: 'elapsed_time_chart.png' and 'operation_counts_chart.png'.")


if __name__ == "__main__":
    run_correctness_suite()
    benchmark_results = run_benchmarks()
    generate_charts(benchmark_results)
```

Appendix C: Link to repo with all python generated charts and code

Link to repo - https://github.com/Tasherokk/Analysis-of-Algorithms_Assignment1.git